

CMP191 – Tendências em Engenharia de Software

Desenvolvimento de um JSON Component Validator com Spec-Driven Development

Aplicando SDD na construção de uma camada determinística para sistemas mHealth baseados em IA.

Autor: Rafael Silveira Machado

Prof.^a Dr.^a Karina Kohl Silveira

Programa de Pós-Graduação em Computação (PPGC)

Universidade Federal do Rio Grande do Sul (UFRGS)

De texto clínico para software



Prontuário / Plano de Cuidados



Gerador baseado em LLM



Componentes JSON (Risco Estrutural)

Informações Clínicas Não Estruturadas

Dados médicos complexos exigem tradução automatizada para artefatos de software consumíveis.

A Natureza Probabilística das LLMs

Modelos generativos podem produzir estruturas inválidas que corrompem o aplicativo mHealth final.

Necessidade central: Uma barreira determinística estrita entre a geração por IA e o consumo pelos dispositivos.

JSON Component Validator

ENTRADA

JSON candidato gerado por LLM

↓ Passa pelo Validador

CAMADA DE DECISÃO

JSON Schema + Template Baseline

✓ **Válido**

Segue adiante para consumo na aplicação mHealth.

✗ **Inválido (Fail-Safe)**

Rejeição imediata (default-deny). Não corrige dados.

"A LLM gera. O validador decide se o resultado pode passar."

Do problema ao software através de especificações



Abordagem adotada: Evitou-se começar diretamente pelo código. O fluxo do **GitHub Spec-Kit** (utilizado via Gemini CLI) garantiu que cada decisão arquitetural estivesse documentada e validada antes de qualquer linha de implementação.

Clarify: resolvendo ambiguidades antes da implementação

QUESTÃO 01

Como o sistema será disponibilizado?

Decisão: API REST padronizada

Permite integração desacoplada com múltiplos frontends e serviços backend.

QUESTÃO 02

Como validar campos estáticos complexos?

Decisão: JSON Schema + Template Baseline

Uso de Recursive Matcher para garantir conformidade estrita de estruturas fixas.

O processo de clarificação evitou retrabalho e definiu com precisão o contrato de validação antes da codificação.

Do plano para o código

TypeScript / Node.js

- ✓ Alinhado ao ecossistema mHealth da pesquisa
- ✓ Tipagem estática forte para contratos de dados
- ✓ Alta produtividade para desenvolvimento de APIs

Stack Tecnológica

Runtime: Node.js 18+

Linguagem: TypeScript 5.x

Framework: Express

Validação: AJV (JSON Schema)

Testes: Jest + Supertest

Resultado: um serviço REST de validação

```
// Endpoint Principal
POST /api/v1/validate

{
  "componentType": "quiz",
  "componentPayload": { ... }
}

// Resposta da API
{
  "isValid": true/false,
  "errors": [ ... ],
  "correlationId": "uuid..."
}
```

Rastreabilidade

Suporte completo a cabeçalhos de correlação (`X-Correlation-ID`) para auditoria.

Organização Modular

Separação clara entre camadas de controle, regras de negócio e utilitários de schema.

Validação rigorosa do próprio validador

18 / 18

Casos de Teste Aprovados

Cobertura de 100% dos cenários especificados

Cenários Validados

- ✓ JSON válido e conformidade com schema
- ✓ Campos obrigatórios ausentes e tipos incorretos
- ✓ Divergência de template baseline
- ✓ Segurança (schemas inexistentes, path traversal)

Integração com a pesquisa de mestrado

Camada de Segurança mHealth

O validador atua como um componente reutilizável na arquitetura da pesquisa, isolando a geração probabilística do consumo determinístico em dispositivos móveis de pacientes.

Garante que nenhum componente corrompido chegue ao aplicativo.

FLUXO CONTÍNUO

1. Nota clínica não estruturada
2. Pipeline LLM / Geração
3. **JSON Component Validator (Filtro)**
4. Componentes conformes no Aplicativo

CONCLUSÃO

O que o trabalho demonstrou?

SDD em Problema Real

Aplicação prática de especificação orientada a modelos de IA e ferramentas modernas.

Processo Estruturado

Especificação → Clarify → Plano → Tarefas → Implementação rigorosa.

Software Testado

18/18 testes aprovados garantindo robustez e conformidade determinística.

Reutilização na Pesquisa

Componente pronto para integrar a arquitetura mHealth do mestrado.

"Mais do que desenvolver um validador JSON, o objetivo foi experimentar um processo onde a especificação orienta a implementação."